
Спецификация

Компонент список

Статус: Черновик

Дата: Ноябрь 8, 2021

Версия: 1.0

Авторы	Компания
Владимир Башев	ПИРФ

Содержание

1. Обзор.....	3
1.1. Введение.....	3
1.2. Примечание.....	3
1.3. Ссылки.....	3
2. Компонент Eco.List1.....	4
2.1. IEcoList1 описание на ECO IDL.....	5
2.1.1. Функция Count.....	6
2.1.2. Функция Item.....	6
2.1.3. Функция Add.....	6
2.1.4. Функция IndexOf.....	6
2.1.5. Функция InsertAt.....	6
2.1.6. Функция Remove.....	6
2.1.7. Функция RemoveAt.....	6
2.1.8. Функция Clear.....	6
2.2. Коды ошибок.....	7
Приложение А: Обучающие программы.....	8

1. Обзор

Данный документ описывает требования к реализации компонента Eco.List1 (Компонент список).

1.1. Введение

Описание.

1.2. Примечание

- Ключевые слова в документе

1.3. Ссылки

Данный параграф содержит ссылки на информацию, помогающую понять данный документ:

[] – наименование ссылки

Доступен по: <http://адрес>

2. Компонент Eco.List1

Компонент, имеет следующие описание:

2.1. IEcoList1 описание на ECO IDL

ECO IDL

```
import "IEcoBase1.h"

[
  object,
  uuid(5AADB4CB4-846C-4576-827B-287B5E67A152),
]

interface IEcoList1 : IEcoUnknown {

  uint32_t          Count          ();

  voidptr_t         Item           ([in] void* value1,
                                     [in] void* value2);

  uint32_t          Add            ([in] void* value1,
                                     [in] void* value2);

  uint32_t          IndexOf        ([in] void* value1,
                                     [in] void* value2);

  void              InsertAt       ([in] uint32_t index,
                                     [in] voidptr_t value);

  void              Remove         ([in] voidptr_t value);

  void              RemoveAt       ([in] uint32_t index);

  void              Clear          ();

}
```

2.1.1. Функция Count

Функция возвращает количество элементов списка.

2.1.2. Функция Item

Функция возвращает элемент списка по индексу.

2.1.3. Функция Add

Функция добавляет элемент в конец списка и возвращает его индекс.

2.1.4. Функция IndexOf

Функция возвращает индекс элемента в списке.

2.1.5. Функция InsertAt

Функция вставляет элемент по указанному индексу.

2.1.6. Функция Remove

Функция удаляет элемент из списка.

2.1.7. Функция RemoveAt

Функция удаляет элемент по индексу.

2.1.8. Функция Clear

Функция очищает список элементов.

2.2. Коды ошибок

Следующая таблица содержит коды ошибок.

Код ошибки	Значение	Описание
ERR_ECO_SUCCESES	0x0000	Выполнено успешно. Ошибок нет.
ERR_ECO_UNEXPECTED	0xFFFF	Непредвиденное условие.
ERR_ECO_POINTER	0xFFEE	Было передано неправильное значение указателя.
ERR_ECO_NOINTERFACE	0xFFED	Такой интерфейс не поддерживается.
ERR_ECO_COMPONENT_NOTFOUND	0xFFE9	Компонент не найден.

Приложение А: Обучающие программы

```

/*
 * <кодировка символов>
 *   Cyrillic (UTF-8 with signature) - Codepage 65001
 * </кодировка символов>
 *
 * <сводка>
 *   EcoList1
 * </сводка>
 *
 * <описание>
 *   Данный исходный файл является точкой входа
 * </описание>
 *
 * <автор>
 *   Copyright (c) 2018 Vladimir Bashev. All rights reserved.
 * </автор>
 *
 */

/* Eco OS */
#include "IEcoSystem1.h"
#include "IdEcoMemoryManager1.h"
#include "IEcoInterfaceBus1.h"
#include "IdEcoList1.h"
#include "IEcoError1.h"

/* Глобальный указатель на системный интерфейс */
IEcoSystem1* g_pISys = 0;

/*
 *
 * <сводка>
 *   Функция EcoMain
 * </сводка>
 *
 * <описание>
 *   Функция EcoMain - точка входа
 * </описание>
 *
 */
int16_t EcoMain(IEcoUnknown* pIUnk) {
    int16_t result = -1;
    /* Указатель на интерфейс работы с системной интерфейсной шиной */
    IEcoInterfaceBus1* pIBus = 0;
    IEcoError1* pIErr = 0;
    /* Указатель на интерфейс работы с памятью */
    IEcoMemoryAllocator1* pIMem = 0;
    char_t ptrErrMsg[100] = {0};
    uint16_t iErrSize = 100;
    char_t* ptrStr = 0;
    char_t* pszStr1 = "Test String 1";

```



```

char_t* pszStr2 = "Test String 2";
/* Указатель на тестируемый интерфейс */
IEcoList1* pIList = 0;
uint32_t iCount = 0;
uint32_t iIndex = 0;

/* Проверка и создание системного интрефейса */
if (g_pISys == 0) {
    result = pIUnk->pVTbl->QueryInterface(pIUnk, &IID_IEcoSystem1, (void **)&g_pISys);
    if (result != 0 && g_pISys == 0) {
        /* Освобождение системного интерфейса в случае ошибки */
        goto Release;
    }
}

/* Получение интерфейса для работы с интерфейсной шиной */
result = g_pISys->pVTbl->QueryInterface(g_pISys, &IID_IEcoInterfaceBus1, (void **)&pIBus);
if (result != 0 || pIBus == 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}

/* Получение интерфейса обработки ошибок */
result = pIBus->pVTbl->QueryInterface(pIBus, &IID_IEcoError1, (void **)&pIErr);
if (result != 0 || pIErr == 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}
pIErr->pVTbl->get_Description(pIErr, (void*)ptrErrMessage, &iErrSize);

#ifdef ECO_LIB
    /* Регистрация статического компонента для работы со списком */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoList1,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_53884AFC93C448ECA929C8D3A562281);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
#endif

/* Получение интерфейса управления памятью */
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoMemoryManager1, 0, &IID_IEcoMemoryAllocator1, (void**)
&pIMem);

/* Проверка */
if (result != 0 || pIMem == 0) {
    /* Освобождение системного интерфейса в случае ошибки */
    goto Release;
}

/* Получение тестируемого интерфейса */
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoList1, 0, &IID_IEcoList1, (void**) &pIList);
if (result != 0 || pIList == 0) {
    iErrSize = 100;
    pIErr->pVTbl->get_Description(pIErr, (void*)ptrErrMessage, &iErrSize);
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

```

```

/* Формирование саиска */
iIndex = pIList->pVTbl->Add(pIList, pszStr1);
iIndex = pIList->pVTbl->Add(pIList, pszStr2);
iCount = pIList->pVTbl->Count(pIList);
/* Проверка количества элементов списка */
if (iCount == 2) {
    for (iIndex = 0; iIndex < iCount; iIndex++) {
        ptrStr = (char_t*)pIList->pVTbl->Item(pIList, iIndex);
    }
    /* Удаление элемента по индексу */
    pIList->pVTbl->RemoveAt(pIList, 0);
    /* Удаление элемента по указателю */
    pIList->pVTbl->Remove(pIList, pszStr2);
}
iCount = pIList->pVTbl->Count(pIList);
/* Формирование списка */
//iIndex = pIList->pVTbl->Add(pIList, pszStr1);
pIList->pVTbl->InsertAt(pIList, 0, pszStr1);
pIList->pVTbl->InsertAt(pIList, 0, pszStr2);
iIndex = pIList->pVTbl->IndexOf(pIList, pszStr1);
/* удаление всех элементов из списка */
pIList->pVTbl->Clear(pIList);
iCount = pIList->pVTbl->Count(pIList);

```

Release:

```

/* Освобождение интерфейса для работы с интерфейсной шиной */
if (pIBus != 0) {
    pIBus->pVTbl->Release(pIBus);
}

/* Освобождение интерфейса работы с памятью */
if (pIMem != 0) {
    pIMem->pVTbl->Release(pIMem);
}

/* Освобождение тестируемого интерфейса */
if (pIList != 0) {
    pIList->pVTbl->Release(pIList);
}

/* Освобождение системного интерфейса */
if (g_pISys != 0) {
    g_pISys->pVTbl->Release(g_pISys);
}

return result;
}

```